



**DE MONTFORT
UNIVERSITY
LEICESTER**

AI Social Lab

AI-POWERED SOCIAL ENGINEERING ATTACKS SIMULATION

Name Here

ROLL NUMBER | BS DEPT

Table of Contents:

Literature Review: 4

 Introduction: 4

 Evolution of Social Engineering Attacks 5

 Artificial Intelligence and Modern Phishing Techniques: 6

 Retrieval-Augmented Generation and Knowledge Integration 7

 Human Factors and Psychological Manipulation 8

 Ethical Considerations in AI-Based Security Training 9

 Research Gap and Project Motivation: 10

 Case Studies: Real-World Social Engineering Failures (2022-2025) 11

 1. The Uber "MFA Fatigue" Breach that happened in September 2022 [13] 11

 2. The MGM Resorts "Vishing" Attack that happened in September 2023 [14] 11

 3. The Arup AI-Deepfake Video Scam that happened in 2024 [15] 11

System Architecture: 12

 UML Use Case Diagram: 13

 System Architecture Diagram: 14

 ER Diagram (Database Schema): 17

 Class Diagram: 19

 Sequence Diagrams: 21

 Authentication Sequence Diagram: 21

 Admin KPI Analytics Sequence Diagram: 22

 AI Message Processing - Flow Chart Diagram: 23

Functional Requirements Specification: 25

 1. Authentication and User Management 25

 FR 1: User Registration 25

 FR 2: User Login 25

 FR 3: Role-Based Access Control 26

 FR 4: Password Reset 26

2. Assessment and Profiling.....	26
FR 5: Personality Assessment.....	26
FR 6: Technical Security Assessment	27
FR 7: Target Profile Generation.....	27
3. Simulation Session Management	27
FR 8: Scenario Selection	27
FR 9: Real-Time Chat Interface	28
FR 10: Persona Introduction and Consent	28
FR 11: Adaptive Tactic Retrieval (RAG).....	28
FR 12: Suspicion Detection and Pivot.....	29
FR 13: Session Termination and Outcome Recording	29
4. AI Orchestration	29
FR 14: Lang Chain Orchestration Loop	29
FR 15: Llama-Based Message Generation	30
FR 16: KPI Computation Engine	30
FR 17: Comprehensive Session Logging	30
FR 18: Admin Dashboard	31
Non-Functional Requirements (NFRs):	31
NFR 1: Performance - Page Load Time.....	31
NFR 2: Performance - LLM Response	31
NFR 3: Availability	32
NFR 4: Scalability	32
NFR 5: Security - Authentication and Authorization	32
NFR 6: Security - Data Encryption	32
NFR 7: Usability - Responsive Design	33
NFR 8: Usability - Accessibility	33
NFR 9: Reliability - Fault Tolerance.....	33
NFR 10: Maintainability.....	34
NFR 11: Compliance - GDPR and Data Privacy	34

NFR 12: Monitoring and Logging	34
Indicative Test Plan:.....	35
Test Objectives:.....	35
Unit Tests:.....	36
Integration Tests:	37
System / End-to-End Tests:	38
Black Box Functional Tests:.....	39
Non-Functional Tests:	40
Implementation:	41
Overview of Prototype Demonstration:.....	41
Front-End Architecture:	41
Back-End and Database Integration:.....	41
AI Orchestration and RAG Pipeline:.....	41
Prototype Scope for First Deliverable:	42
References:	46

Literature Review:

Introduction:

People have always been a part of keeping things safe online. Companies spend a lot of money on things like firewalls and encryption to keep their information safe. Bad people often find ways to get around these safety measures by tricking people instead of trying to break through the safety walls. This is called engineering.

Social engineering is when someone tricks you into doing something that's not safe by playing on your feelings like trust or fear. They want you to give them information or do something that will let them get into your system.

Lately it has become clear that we need to make sure people are aware of the dangers. There have been big problems with safety because of simple tricks like phishing. These problems show that even companies with safety measures can still have issues if their employees are not taught how to spot tricks.

At the time recent technologies are changing how we keep things safe online. New computer programs can make text that looks like it was written by a person. This means bad people can use these programs to send fake messages. These same programs can also be used to help people learn about safety by making practice situations that feel real.

This project is, about using computer programs that can talk like people to practice social engineering tricks in a place. The program uses language models and knowledge systems to make situations that're realistic and help people learn to spot tricks. To understand why this is an idea we need to look at how social engineering tricks have changed how recent technologies are affecting phishing how language models can help and if it is okay to use fake tricks to instruct people.

Evolution of Social Engineering Attacks

Social engineering is not a new concept. Long before the emergence of digital communication networks, attackers relied on deception and impersonation to obtain confidential information. Early examples often involved telephone calls in which attackers pretended to be technical support staff or company administrators. By exploiting trust and authority, they were sometimes able to persuade employees to reveal passwords or internal procedures.

As digital communication became more widespread, social engineering techniques adapted accordingly. Email rapidly became one of the most common channels for deceptive communication. Phishing emails attempt to convince recipients that they are interacting with a legitimate organization or colleague. Victims are typically encouraged to click malicious links, download infected attachments, or provide login credentials through fraudulent websites. Early phishing campaigns were relatively unsophisticated. Messages often contained poor grammar, unusual formatting, or suspicious sender addresses. As a result, attentive users could often identify them without much difficulty.

Over time, however, attackers refined their techniques. Many modern phishing messages closely resemble legitimate corporate communications. Logos, formatting styles, and writing tone may all be carefully replicated to create the impression of authenticity. Researchers have also observed an increase in targeted social engineering attacks. Instead of sending generic messages to large numbers of recipients, attackers sometimes conduct preliminary research about their targets. Public information available on social networking platforms or corporate websites can reveal job roles, internal departments, and communication patterns. This information enables attackers to craft personalized messages that appear highly relevant to the recipient.

One well-known example of this approach is the business email compromise attack. In these cases, attackers impersonate senior executives or financial managers within an organization. They may request urgent financial transfers, confidential data, or internal documents. Because the messages appear to originate from trusted authority figures, employees may comply without performing additional verification. Another important development is the shift toward interactive communication. Traditional phishing attacks typically involved a single message followed by a malicious link. Modern social engineering campaigns may instead involve multiple exchanges between the attacker and the victim.

Through these conversations, attackers gradually build trust before introducing requests that compromise security. This conversational nature makes such attacks more difficult to detect, as the interaction resembles normal workplace communication.

Artificial Intelligence and Modern Phishing Techniques:

Artificial intelligence is getting better. This is changing the way people commit cybercrime and the way we defend against it. Some computers, like GPT-4 and Llama 3.1 are good at understanding what we say and writing back in a way that makes sense. They can even write sentences that're grammatically correct.

If someone wants to hack into our computers, they can use intelligence to help them. They can make emails that look real and sound like they are coming from someone we know. Artificial intelligence can look at what we like and what we do. Then author an email that seems like it is just for us. This is called phishing. It is very sneaky.

The sad thing about intelligence and phishing is that it can send a lot of emails amazingly fast. Usually when someone tries to trick us, they must author each email themselves. With artificial intelligence they can make thousands of emails in just a few minutes. This means they can try to trick a lot of people at the time and still make each email seem personal. Artificial intelligence is making phishing a problem for cybersecurity defence, and we need to be careful, about artificial intelligence and phishing.

Researchers have also found that conversational AI systems can be used to keep talking to victims. The AI system does not just send one message it talks back to the victim. Changes what it says as the conversation goes on. This makes the attack seem real. It is more likely that the victim will be tricked.

The use of intelligence has some downsides, but it can also be used to help keep people safe. Some security experts think that AI can be used to practice what a real attack would be like. This helps people learn what to do if they are being attacked. By using AI to simulate attacks people can learn how to spot something that does not seem right and know what to do about it in a safe place.

There are some problems, with using AI to train people. One big problem is that AI systems can sometimes say things that're not true. This is called hallucination. It happens when the AI system says something that sounds okay. Is not based on facts. If this happens when people are being trained to spot cyber attacks it can make the training seem real and less useful.

Retrieval-Augmented Generation and Knowledge Integration

To deal with the problems of language models that work alone researchers have come up with ways to combine these models with sources of information. One fashionable way to do this is called retrieval-augmented generation. This method involves the system finding information in a database before it gives a response.

The system uses something called embeddings to find this information. It takes text. Turns it into numbers that show what the text means. When someone asks a question, the system turns the question into an embedding too. Then it looks for embeddings in the database.

There are databases like Pinecone that are good at finding similar embeddings. They make it easy to find the information in a big collection of embeddings. Once the system finds the information it gives it to the language model to use. The model then uses this information to give a response.

This way of doing things has a lot of advantages when you need to get relevant information. The model is less likely to make things up because it is using information from sources. Also, the database can be updated without having to retrain the model. This means the system can learn things without having to start over.

For example, in cybersecurity training retrieval-augmented generation can be used to simulate social engineering attacks. The system does not just use what it already knows. Instead, it finds attack strategies in a database and uses them to create messages that are tailored to the user. The language model generates messages that use these strategies and are relevant to the user. This makes the training more realistic and effective.

Human Factors and Psychological Manipulation

Understanding psychology is important when we look at how well social engineering attacks work. A lot of attacks use the fact that people behave in predictable ways rather than exploiting technical weaknesses. Things like making people feel urgent or making them think someone important is telling them to do something or making them think they will miss out if they do not act fast or just making them curious are often used in messages that are trying to trick people.

For example, messages that say something must be done now might make people act quickly without checking if the request is real. Similarly, messages that seem to come from someone at work might make employees do what they are told because they respect the persons position. People who launch these attacks often use several of these tricks at the same time.

Research into how people behave when it comes to cybersecurity says that training programs should teach people to spot not suspicious technical things but also how to recognize when someone is trying to manipulate their emotions. Social engineering attacks and psychological manipulation are problems and social engineering attacks are a type of psychological manipulation. People who understand how social engineering attacks and psychological manipulation work are better able to deal with them.

Places where people can practice dealing with social engineering attacks can be helpful. These systems let people see how social engineering attacks and psychological pressure can change the way they behave. Then they get feedback on why some things they did might have made them more vulnerable to social engineering attacks and psychological manipulation. Social engineering attacks are a type of attack that uses manipulation and understanding how social engineering attacks work is key to stopping them.

Ethical Considerations in AI-Based Security Training

Artificial intelligence simulation platforms are helpful for learning. We need to think about the ethics of using them. When we use intelligence to simulate behaviour, we must be careful about how we design it so that it does not get used in a bad way. One thing we must do is make sure the system only works in a training area.

We should not use email or messaging to practice attacks. This way we can stop the system from being used to trick people into giving away information. Another thing to think about is keeping user information private. When people use these training systems, they often collect data about how users behave, like how they react and what they decide.

The people in charge must make sure this data is kept safe and only used to help people learn. We also must be open about how we collect and use this data for intelligence simulation platforms.

Artificial intelligence simulation platforms can also affect people's feelings. If the simulations are too realistic or scary people might get upset or worried.

That is why some training programmes tell people that it was a simulation and give them feedback about what they did, with intelligence simulation platforms. This helps people learn from the intelligence simulation platforms without getting too stressed.

Research Gap and Project Motivation:

The thing about phishing detection and security awareness training is that a lot of work has been done on phishing detection and security awareness training. The problem is that many of the ways people do phishing detection and security awareness training now are not very good at copying what really happens when someone tries to trick us with phishing attacks. These people who try to trick us with phishing attacks are very good at talking to us and changing what they do to catch us off guard with phishing attacks.

The idea of using computers that can talk to us and remember things that have happened before with phishing detection and security awareness training is great. If we use computers that can understand what we say and have a list of things that bad people have done before with phishing attacks we can make situations that're like real life phishing attacks. This way we can learn about phishing detection and security awareness training in a way that's fun and feels real.

This project is trying to make a system like that using the internet for phishing detection and security awareness training. When we answer questions from phishing attacks the system tries to figure out how well we can spot the phishing tricks and how much we know about being safe from phishing attacks. This can help people who are in charge see where we need to learn more about being safe, from phishing attacks and phishing detection and security awareness training.

Case Studies: Real-World Social Engineering Failures (2022-2025)

Here are some real-life examples that show how social engineering can get around security measures by exploiting people. This is why we need training platforms that can adapt to situations like the one we are proposing in this project.

1. The Uber "MFA Fatigue" Breach that happened in September 2022 [13]

The way they did it: They used a combination of trying passwords and wearing down the person with MFA notifications.

What happened: Someone who was linked to the Lapsus\$ group bought an Uber contractors login information on the web. When they could not get past the security check, they just kept sending notifications to the employee until they got fed up and accepted one.

What was the result: The bad person got access to Ubers internal messaging system, cloud storage and Google apps.

Why it matters: This shows that with extra security checks people are still the weak link. If someone does not know about "MFA Bombing" tactics they can still get tricked. This is something that regular training videos do not do a job of teaching.

2. The MGM Resorts "Vishing" Attack that happened in September 2023 [14]

The way they did it: They used voice phishing. Pretended to be someone else.

What happened: Some bad people from "Scattered Spider" found an MGM employees name and job title on LinkedIn. Then they called the IT help desk. Said they were that employee asking to reset their password. In ten minutes, they got control of the system.

What was the result: The attack cost MGM around \$100 million. It stopped their slot machines, hotel room keys and booking system from working for days.

Why it matters: This shows how powerful it can be when someone manipulates another person. It also shows that the people who work at the IT help desk are just as much at risk, as employees.

3. The Arup AI-Deepfake Video Scam that happened in 2024 [15]

The way they did it: They used fake videos and audio that were made by a computer.

What happened: Someone who works with money at the engineering company Arup was tricked into a video meeting where everyone else including the financial officer was a fake made by a computer. They were told to send \$25 million to accounts.

What was the result: The money was sent before the employee realized that the whole meeting was a realistic simulation.

Why it matters: This is an example of a kind of phishing that uses artificial intelligence. It makes the old advice to "look for flags" not work anymore. This is why we want to use Llama 3.1 in our project to help people get ready for frauds that use intelligence and are very convincing.

System Architecture:

The system follows a layered architecture that separates user interaction, application logic, artificial intelligence orchestration, and data storage components. This approach improves scalability, maintainability, and security.

The frontend interface is implemented using Next.js, which provides both client-side rendering and server-side functionality through server actions. These server actions function as API endpoints responsible for handling authentication, session management, and communication with backend services.

The orchestration layer is implemented using Lang Chain. This part coordinates the interaction between the language model and the retrieval system. When a user message is received, the orchestrator analyses the content and decides which social engineering tactic should be retrieved.

Relevant tactics are retrieved from the vector database using Pinecone. Each tactic is stored as an embedding that allows efficient similarity search. The retrieved tactics are then included in the prompt sent to the language model.

The conversational responses are generated by the Llama 3.1 language model. This model produces contextually proper attacker messages that simulate realistic social engineering behaviour.

All session data, user information, and analytics are stored in MongoDB. This database also records user interactions and KPI events that are later analysed through the administrative dashboard.

UML Use Case Diagram:

The use case diagram illustrates the primary interactions between system users and the AI-driven social engineering training platform. Two main actors interact with the system: the **Employee**, who participates in simulated training sessions, and the **Security Officer (Administrator)**, who monitors system performance and user behavior.

This diagram represents a **bridge between a user (Employee) and an AI (Attacker)** to test security awareness.

1. The Core Interaction (The "Brain")

The most important part of the diagram is the **Interact with AI Attacker** section.

- It shows that every time an employee speaks, the system must **Retrieve Social Engineering Tactics** (the "strategy") and **Generate AI Response** (the "speech").
- Think of this as the AI "thinking" before it "speaks" to try and trick the employee.

2. The Feedback Loop

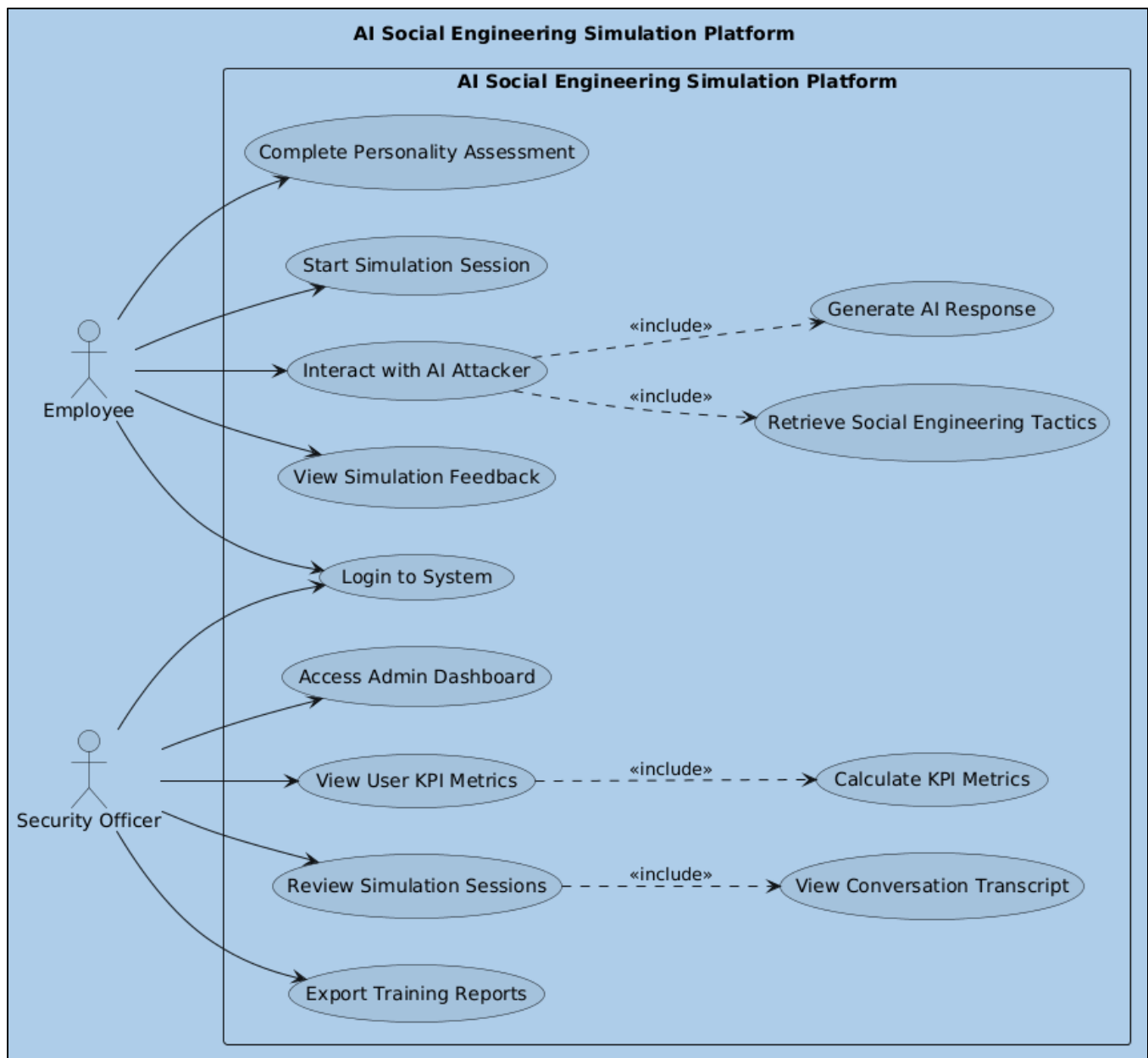
The diagram shows a clear split between **Action** and **Analysis**:

- **Employee Side:** They do the "work" (Assessments, Simulations).
- **Security Officer Side:** They do the "oversight." They don't just see a "Pass/Fail" grade; they can drill down into **Conversation Transcripts** to see exactly where an employee was vulnerable.

3. The Functional Flow (Simplified)

If you follow the arrows, the process looks like this:

1. **Preparation:** Employee takes a personality test.
2. **Execution:** Employee chats with the AI (which uses LLM and RAG tactics).
3. **Reporting:** The system calculates KPIs (Key Performance Indicators).
4. **Review:** The Security Officer exports a report for the company.



System Architecture Diagram:

This diagram is a **System Architecture Diagram** that shows how the platform is actually built. It explains how data flows from the user's screen into the AI "brain" and back.

Here is the high-level breakdown of each layer:

1. User Interface (The Frontend)

- **Employee Chat UI:** Where the trainee talks to the AI.
- **Admin Dashboard:** Where the security officer sees the analytics.
- Both communicate via **API Endpoints (Server Actions)** in a **Next.js** environment.

2. Orchestration Layer (The "Brain")

- **LangChain.js Orchestrator:** This is the most important part. It manages "Agent Loops." Instead of just sending a message to an AI, it decides *how* to handle the message—checking history, pulling tactics, and then deciding what to say next.

3. Data & AI Layer (The "Memory")

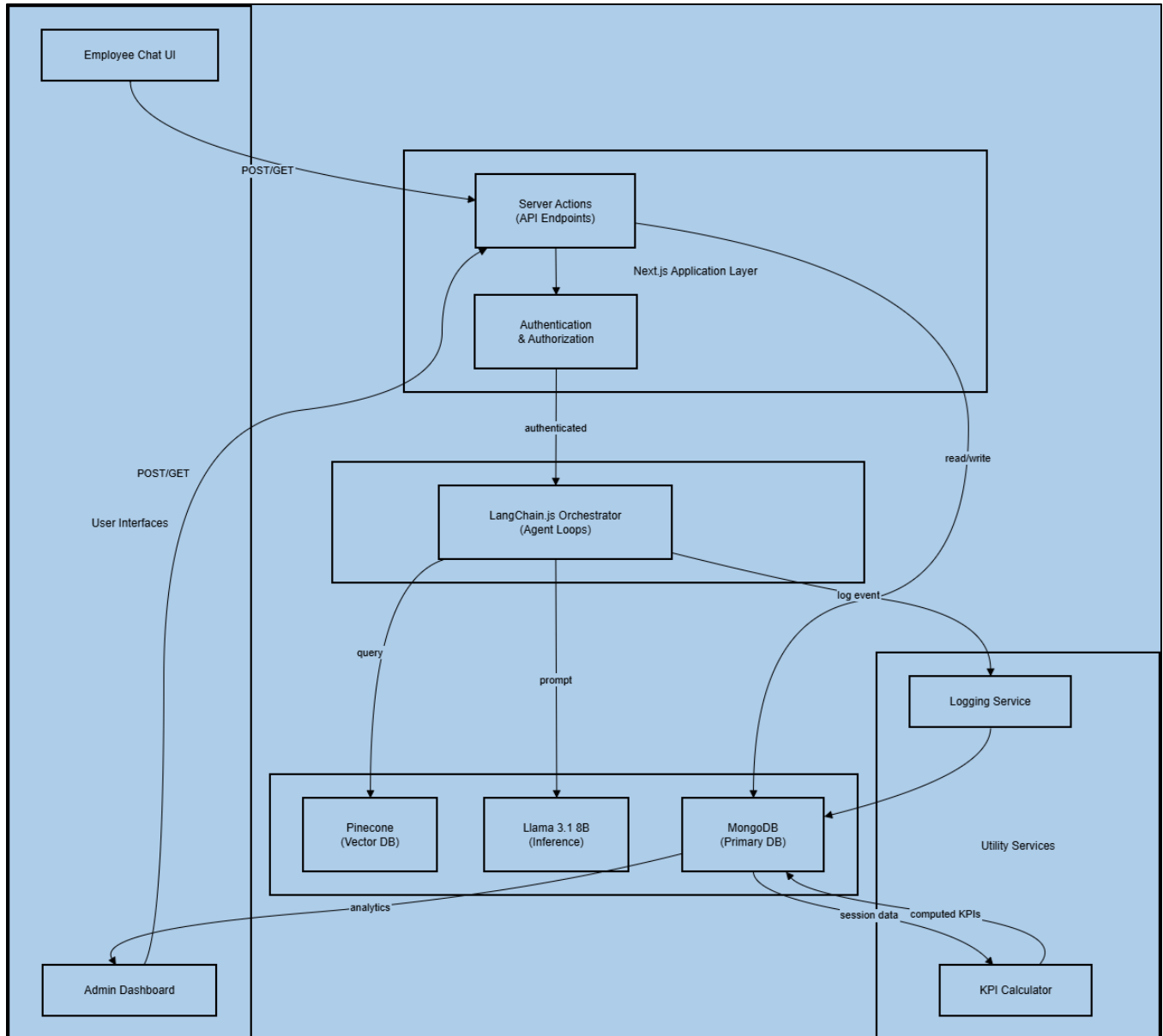
- **Pinecone (Vector DB):** This stores the social engineering tactics. It allows the AI to "search" for a specific strategy (like phishing) based on the conversation context (**RAG - Retrieval Augmented Generation**).
- **Llama 3.1 8B (Inference):** This is the actual Large Language Model (LLM) that generates the human-like text responses.
- **MongoDB (Primary DB):** This stores standard data like user accounts, profiles, and simulation logs.

4. Utility Services (The "Analytics")

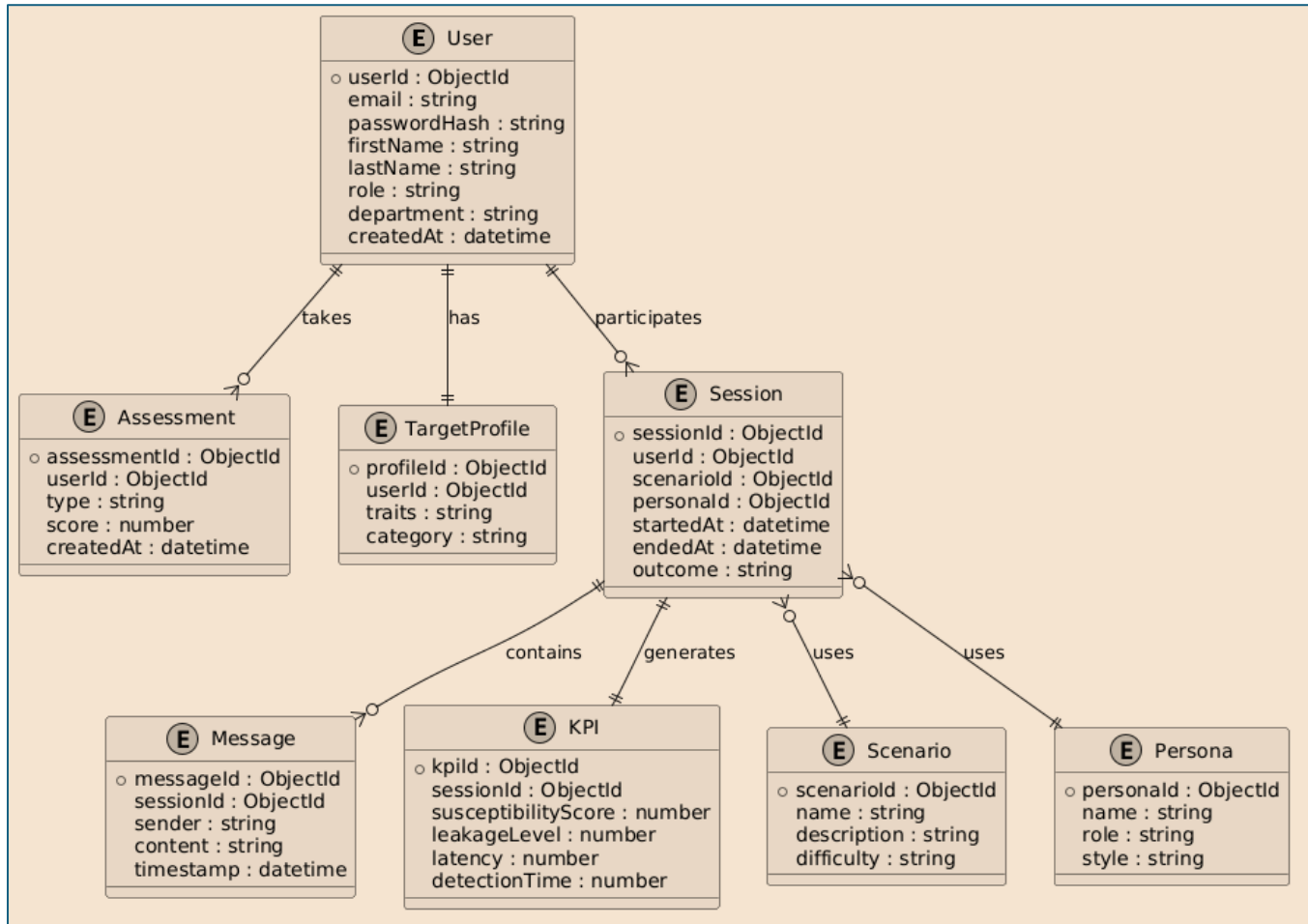
- **Logging Service:** Records every event for security and review.
- **KPI Calculator:** Takes the "Session Data" from MongoDB and calculates the scores (Key Performance Indicators) that the Admin sees on their dashboard.

Simplified Flow of a Message:

1. **Employee** sends a message in the **Chat UI**.
2. **Next.js** sends it to the **LangChain Orchestrator**.
3. **LangChain** queries **Pinecone** for a tactic and sends a prompt to **Llama 3.1**.
4. The response is logged in **MongoDB** and sent back to the **Employee**.
5. The **KPI Calculator** later analyzes that log to update the **Admin Dashboard**.



ER Diagram (Database Schema):



ER Diagram Description

The ER diagram represents the database structure of the AI-driven social engineering training platform. It models the key entities and their relationships required to manage users, simulations, and analytics.

Core Entities

1. User

Represents both trainees and administrators.

Stores authentication and profile information such as email, role, and department.

2. Assessment

Stores results of personality and technical assessments.

Each user can have multiple assessments over time.

Relationship:

User → Assessment = One-to-Many

3. Target Profile

Represents the vulnerability profile generated from assessments.

Each user has one active target profile.

Relationship:

User → Target Profile = One-to-One

4. Session

Represents a simulation session between a trainee and the AI attacker.

Stores scenario, persona, timestamps, and outcome.

Relationship:

User → Session = One-to-Many

5. Message

Stores chat messages exchanged during a session.

Includes both user and AI-generated messages.

Relationship:

Session → Message = One-to-Many

6. KPI

Stores computed performance metrics after a session.

Includes susceptibility score, leakage level, latency, and detection time.

Relationship:

Session → KPI = One-to-One

7. Scenario

Defines the type of social engineering attack (e.g., CEO fraud, phishing).

8. Persona

Defines the attacker's behavior and communication style.

Relationships Summary

- A **User** can have multiple **Sessions**
- A **Session** uses one **Scenario** and one **Persona**
- A **Session** contains multiple **Messages**
- A **Session** generates one **KPI record**
- A **User** has one **Target Profile**

Class Diagram:

Class Diagram Description

The class diagram represents the internal structure of the system using object-oriented design principles. It defines the main classes, their attributes, methods, and relationships.

Core Classes

1. User

Represents system users (trainee or admin).

Handles authentication and account management through methods like `register()`, `login()`, and `resetPassword()`.

2. Assessment

Stores results of personality and technical tests.

Includes logic to calculate scores.

3. TargetProfile

Represents the behavioral profile derived from assessments.

Used to personalize AI-driven simulations.

4. Session

Represents a simulation session.

Manages lifecycle using `startSession()` and `endSession()`.

5. Message

Represents individual chat messages exchanged between user and AI.

Each session contains multiple messages.

6. KPI

Stores performance metrics such as susceptibility score and leakage level.

Generated after session completion using `computeKPIs()`.

7. Scenario

Defines the attack scenario (e.g., phishing, CEO fraud).

8. Persona

Defines attacker characteristics such as tone, role, and communication style.

9. AIEngine

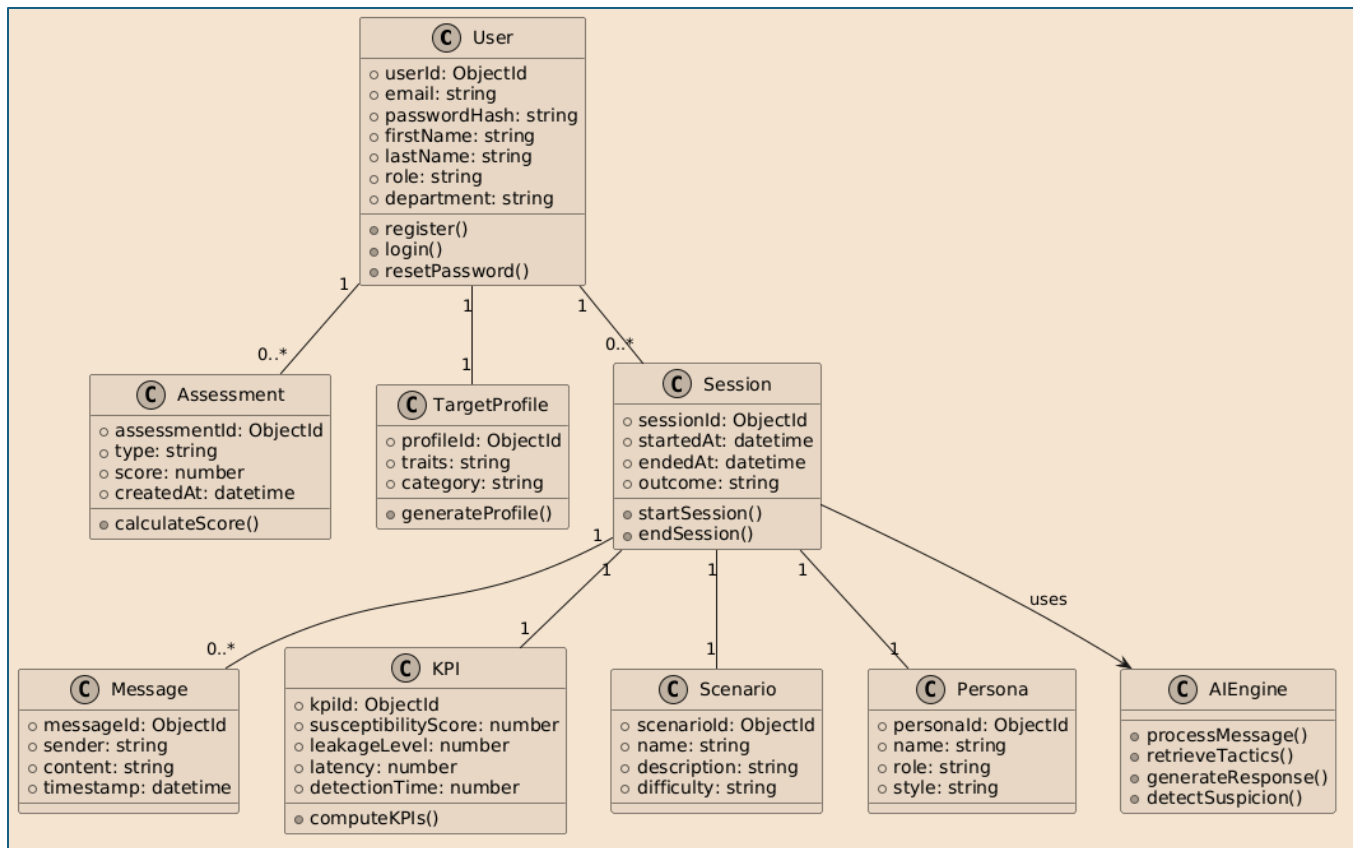
Represents the AI orchestration logic.

Handles:

- Message processing
- Tactic retrieval (RAG)
- Response generation (LLM)
- Suspicion detection

Relationships

- A **User** can have multiple **Sessions** and **Assessments**
- A **Session** contains multiple **Messages**
- A **Session** is linked to one **Scenario** and one **Persona**
- A **Session** generates one **KPI record**
- The **AIEngine** is used by the Session to process interactions

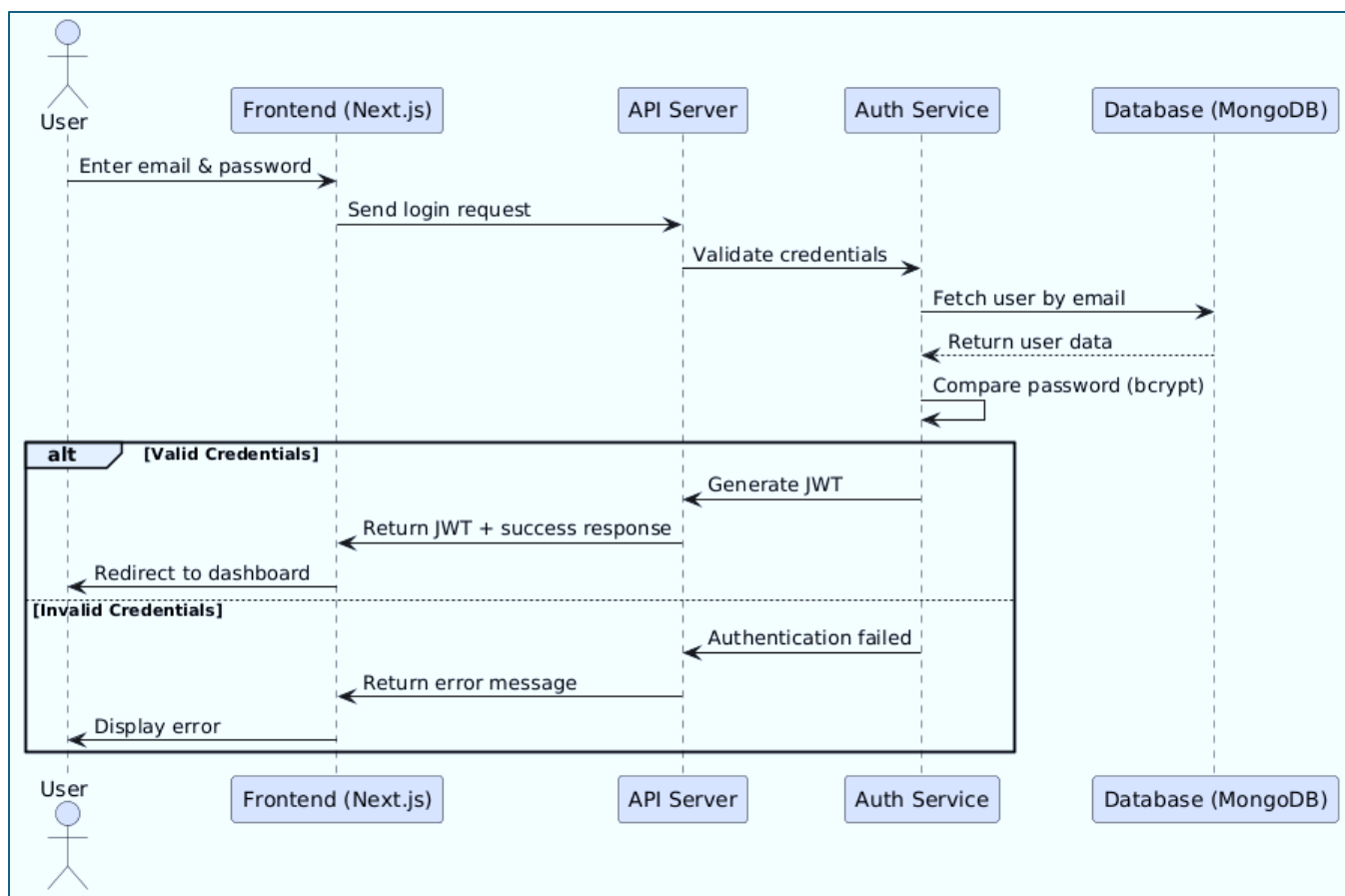


Sequence Diagrams:

Authentication Sequence Diagram:

The authentication sequence diagram represents the step-by-step interaction between the user and system components during the login process. When a user submits their email and password through the frontend interface, the request is sent to the backend server (API layer). The system then validates the credentials by comparing the provided password with the stored hashed password in the database. If the credentials are valid, a JSON Web Token (JWT) is generated and returned to the client, allowing the user to access protected resources such as the dashboard.

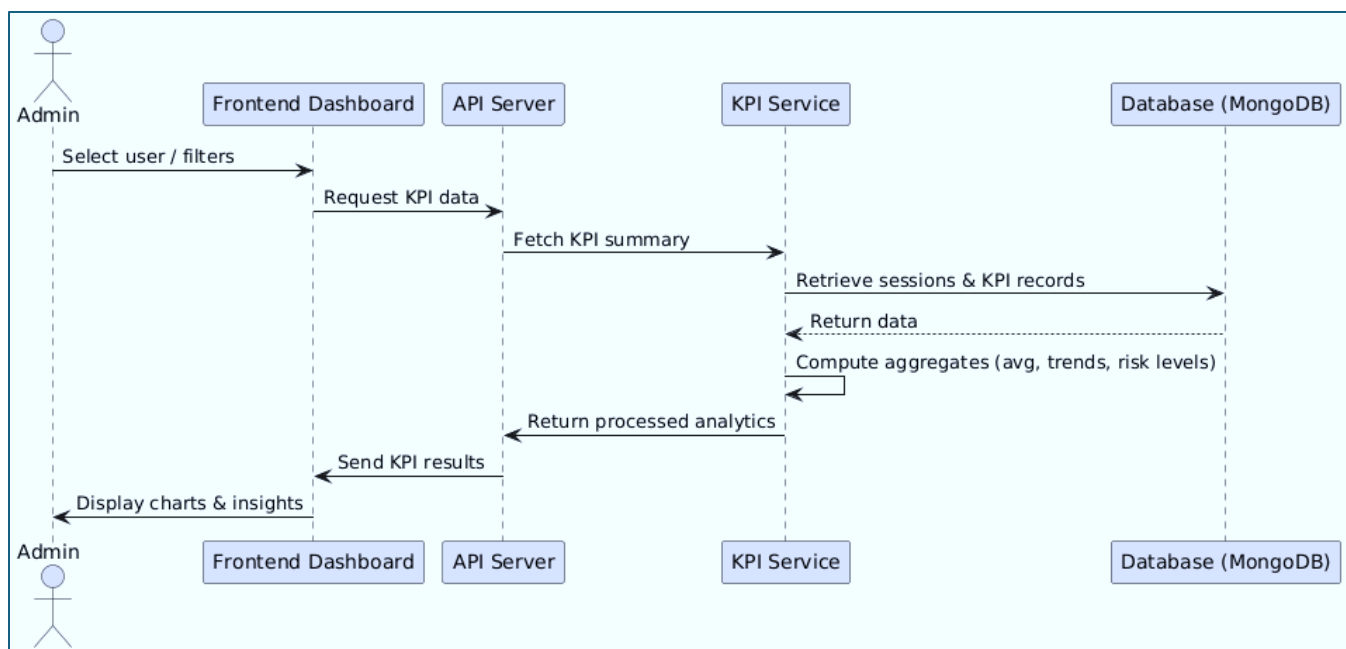
If the credentials are invalid, the system returns an error response and logs the failed attempt for security monitoring. Additional security measures, such as account lockout after multiple failed attempts, are enforced at this stage. Overall, the process ensures that only authenticated users can access the platform while maintaining secure session management and protecting against unauthorized access.



Admin KPI Analytics Sequence Diagram:

The Admin KPI Analytics sequence diagram illustrates how the system retrieves and processes performance data for display on the administrative dashboard. When an administrator selects a user or applies filters, the frontend sends a request to the backend API. The API forwards this request to the KPI service, which retrieves relevant session data and previously computed KPI records from the database.

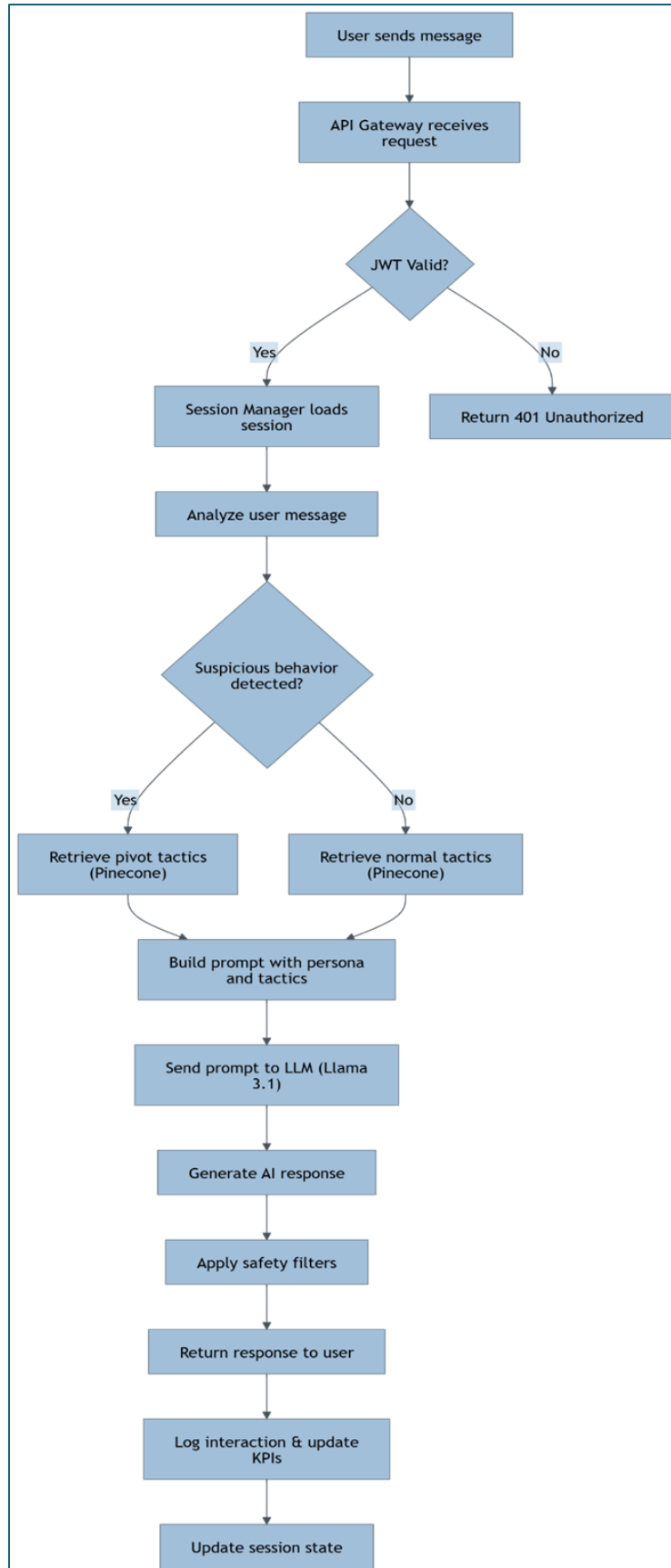
The KPI service then processes this data to compute aggregated metrics such as average susceptibility scores, trends over time, and risk classifications. These processed analytics are returned to the API and then sent to the frontend, where they are visualized as charts and insights for the administrator. This process enables efficient monitoring of user performance and supports data-driven security training decisions.



AI Message Processing - Flow Chart Diagram:

The AI Message Processing flowchart illustrates the internal workflow followed when a user sends a message during a simulation session. The process begins with the API validating the user's authentication token and retrieving the active session. The system then analyzes the user's message to determine intent and detect any signs of suspicion. Based on this analysis, the system retrieves appropriate social engineering tactics from the vector database, either continuing the attack strategy or pivoting in response to user skepticism.

The retrieved tactics, along with persona guidelines and conversation history, are used to construct a prompt for the language model. The LLM generates a context-aware response, which is then passed through safety filters before being returned to the user. Finally, the system logs the interaction, updates session data, and records KPI-related metrics, enabling continuous tracking of user behavior and performance throughout the simulation.



Functional Requirements Specification:

1. Authentication and User Management

FR 1: User Registration

Priority: Critical

Actor: New User

The system shall allow new users to create an account by providing email, password, first name, last name, and department.

Acceptance Criteria:

- Validate email format (RFC 5322).
- Enforce password strength: ≥8 characters, 1 uppercase, 1 lowercase, 1 number, 1 special character.
- Prevent duplicate emails (case-insensitive).
- Store verified accounts in MongoDB.
- Send confirmation email with verification link.

FR 2: User Login

Priority: Critical

Actor: Registered User

The system shall authenticate users using email and password.

Acceptance Criteria:

- Validate credentials against stored bcrypt hashes.
- Issue JWT valid for 8 hours upon successful login.
- Lock account for 30 minutes after 5 consecutive failed attempts.
- Provide error messages for invalid login attempts.

FR 3: Role-Based Access Control

Priority: Critical

Actor: All Users

The system shall enforce role-based access control for trainee and admin functionality.

Acceptance Criteria:

- Trainees cannot access admin routes (/dashboard, /scenarios, /personas).
- Admins cannot access trainee simulation routes.
- Log and reject unauthorized attempts with 403 responses.

FR 4: Password Reset

Priority: High

Actor: Registered User

The system shall offer a self-service password reset mechanism.

Acceptance Criteria:

- Generate 24-hour time-limited reset tokens.
- Send reset link via email.
- Allow user to enter a new password meeting strength requirement.
- Invalidate old reset tokens after use.

2. Assessment and Profiling

FR 5: Personality Assessment

Priority: Critical

Actor: Trainee

The system shall administer a 50-question Big Five (OCEAN) personality assessment.

Acceptance Criteria:

- Score each dimension 0–100.
- Require all questions to be answered before going ahead.
- Store results in MongoDB with timestamp.
- Provide trainee feedback and interpretation.

FR 6: Technical Security Assessment

Priority: High

Actor: Trainee

The system shall administer a 20-question multiple-choice security awareness quiz.

Acceptance Criteria:

- Score each answer; total score 0–20.
- Display results at once with explanations for incorrect answers.
- Track historical attempts and recommend training modules for weak areas.

FR 7: Target Profile Generation

Priority: Critical

Actor: System

The system shall generate a Target Profile based on assessment results.

Acceptance Criteria:

- Map Big Five and technical scores to profile categories.
- Store profiles in MongoDB for session personalization.
- Support at least 8 distinct profile types.

3. Simulation Session Management

FR 8: Scenario Selection

Priority: Critical

Actor: Trainee

The system shall allow trainees to select a scenario and attacker persona to begin a session.

Acceptance Criteria:

- Display available scenarios and personas.
- Initialize a new session document in MongoDB with timestamps.
- Load trainee Target Profile for adaptive simulation.

FR 9: Real-Time Chat Interface

Priority: Critical

Actor: Trainee

The system shall provide a chat interface for trainee-attacker interaction.

Acceptance Criteria:

- Display conversation history.
- Trigger AI response within 2.5 seconds.
- Support message input with line breaks and emojis.
- Limit sessions to 50 turns; allow manual termination.

FR 10: Persona Introduction and Consent

Priority: High

Actor: Trainee

The system shall show the attacker persona and scenario goal before starting the session.

Acceptance Criteria:

- Trainee must explicitly consent to take part.
- Log consent or decline in KPI Events.

FR 11: Adaptive Tactic Retrieval (RAG)

Priority: Critical

Actor: System

The system shall retrieve persona-aligned social engineering tactics from Pinecone.

Acceptance Criteria:

- Use profile tags and persona filters for retrieval.
- Provide fallback to local tactic library if Pinecone unavailable.
- Display retrieved tactics in event log.

FR 12: Suspicion Detection and Pivot

Priority: High

Actor: System

The system shall detect trainee suspicion and adjust attacker tactics accordingly.

Acceptance Criteria:

- Detect suspicion via keyword/NLP confidence >0.7.
- Retrieve pivot tactics from Pinecone to address user concerns.
- Log pivot events for KPI analysis.

FR 13: Session Termination and Outcome Recording

Priority: High

Actor: Trainee or System

The system shall record session termination and classify outcome.

Acceptance Criteria:

- Record session end reasons: manual abort, max turns, detection, disclosure.
- Assign outcome: attack_success, attack_partially_success, attack_failed, user_aborted.
- Trigger KPI computation.

4. AI Orchestration

FR 14: Lang Chain Orchestration Loop

Priority: Critical

Actor: System

The system shall implement a React-style loop: Retrieve → Generate → Evaluate → Log.

Acceptance Criteria:

- Complete each loop in <2.5 seconds.
- Maintain conversation state in MongoDB.
- Include evaluation for suspicion and leakage.

FR 15: Llama-Based Message Generation

Priority: Critical

Actor: System

The system shall generate attacker messages using Llama 3.1 8B.

Acceptance Criteria:

- Condition messages on persona, tactics, last 5 turns.
- Enforce content constraints (no real credentials, no external links).
- Limit messages to <500 characters.

FR 16: KPI Computation Engine

Priority: Critical

Actor: System

The system shall compute KPIs after session completion.

Acceptance Criteria:

- Compute Susceptibility Score (0–100), Leakage Level (0–4), Response Latency, Time to Detection.
- Store results in KPI Summaries.
- Provide feedback to trainee and admin.

FR 17: Comprehensive Session Logging

Priority: Critical

Actor: System

The system shall log all session interactions and KPI events.

Acceptance Criteria:

- Include timestamps, user messages, attacker responses, latency, suspicion indicators, tactic references.
- Encrypt logs at rest.
- Retain for 12 months.

FR 18: Admin Dashboard

Priority: High

Actor: Admin

The system shall provide a dashboard for KPI analytics and user management.

Acceptance Criteria:

- Display aggregated KPIs, trends, and high-risk cohorts.
- Allow filtering by department, persona, scenario.
- Support CSV/PDF export of analytics and reports.

Non-Functional Requirements (NFRs):

NFR 1: Performance - Page Load Time

Priority: High

The system shall ensure that web pages load fully within 2 seconds under normal load conditions.

Rationale: Fast load improves trainee engagement and reduces frustration.

Acceptance Criteria:

- First content full paint < 2 seconds.
- Chat interface ready to accept input at once after page load.

NFR 2: Performance - LLM Response

Priority: High

The system shall deliver AI-generated responses in the chat interface within 2.5 seconds (p95 latency).

Acceptance Criteria:

- Lang Chain orchestration loop completes < 2.5 seconds.
- Users do not experience noticeable lag in conversation flow.

NFR 3: Availability

Priority: High

The system shall keep at least 99% uptime during business hours (8 AM–6 PM UTC).

Acceptance Criteria:

- Planned maintenance $\leq$ 2 hours/month.
- Unplanned downtime $\leq$ 1 hour/month.
- Core chat and assessment functions stay operational during partial outages.

NFR 4: Scalability

Priority: Medium

The system shall support concurrent users and scale dynamically.

Acceptance Criteria:

- Minimum 50 simultaneous sessions in Phase 1.
- Load balancing across API servers.
- Auto-scaling MongoDB and LLM inference queueing.

NFR 5: Security - Authentication and Authorization

Priority: Critical

The system shall secure user access using role-based authentication and JWT tokens.

Acceptance Criteria:

- JWT tokens expire in 8 hours.
- Passwords stored as bcrypt hashes.
- Unauthorized access attempts logged and rejected with 403 responses.

NFR 6: Security - Data Encryption

Priority: Critical

The system shall encrypt sensitive data at rest and in transit.

Acceptance Criteria:

- MongoDB encryption-at-rest enabled.

- TLS 1.2+ for all HTTP communication.
- Logs having sensitive information are encrypted.

NFR 7: Usability - Responsive Design

Priority: High

The system shall support desktop, tablet, and mobile devices with consistent user experience.

Acceptance Criteria:

- Minimum touch target size: 44×44 px on mobile.
- Landscape and portrait orientations supported.
- No horizontal scrolling on mobile devices.

NFR 8: Usability - Accessibility

Priority: Medium

The system shall follow WCAG 2.1 AA accessibility standards.

Acceptance Criteria:

- Screen reader support via semantic HTML and ARIA labels.
- Keyboard navigation for all functions.
- Colour contrast $\geq 4.5:1$ for text.

NFR 9: Reliability - Fault Tolerance

Priority: High

The system shall manage service interruptions gracefully without data loss.

Acceptance Criteria:

- If Pinecone retrieval fails, fallback to local tactic library.
- Chat session state restored after temporary connection drop.
- Automatic retries with exponential backoff for transient failures.

NFR 10: Maintainability

Priority: Medium

The system shall be modular and maintainable to support future enhancements.

Acceptance Criteria:

- Codebase follows clean architecture principles.
- APIs are well-documented and versioned.
- Logs and metrics allow troubleshooting without code changes.

NFR 11: Compliance - GDPR and Data Privacy

Priority: Critical

The system shall follow GDPR and organizational data privacy policies.

Acceptance Criteria:

- Explicit user consent obtained before registration.
- Users can export or cut their data.
- Data retention policies enforced (default 12 months).
- Audit logs append-only and timestamped.

NFR 12: Monitoring and Logging

Priority: High

The system shall provide monitoring for performance, errors, and security events.

Acceptance Criteria:

- Real-time alerts for critical errors or security events.
- Session logs include latency, suspicion indicators, and KPI events.
- Dashboard shows system health metrics (uptime, API response time, LLM queue length).

Indicative Test Plan:

Testing is an integral part of the software development life cycle; it is the investigation about a feature or set of features to ensure the quality of product and ensure meets the expected requirements. Given the scale of this project, a large series of tests of both the engine and sandbox will be needed to ensure all functional and non-functional requirements are met. The following set of paragraphs will discuss my testing strategy, the set of test cases with descriptions of what was testing and the data and the corresponding result of those tests.

Test Objectives:

The project will organise the design of the software with the object-oriented programming model (OOP). There are many benefits for using the OOP model such as reusability, modularity, robustness, etc to name a few. This provides a clear path for testing as I can evaluate these different categories and with the project made up of modules, I can evaluate individual sections of the code without relying on others. I will be performing a variety of test methodologies such as unit testing and black box testing.

For my system, i have categorized the test cases into five types.

A typical division could be:

1. **Unit Tests** - Testing individual components, small code units (e.g., login validation, password hashing).
2. **Integration Tests** - Testing interactions between components (e.g., chat pipeline: user message → RAG → LLM → response).
3. **System Tests / End-to-End Tests** - Testing full user workflows (e.g., completing an assessment, running a simulation).
4. **Black Box Tests** - Focus on functional behaviour without internal knowledge (e.g., scenario choice, persona consent).
5. **Non-Functional Tests** - Performance, availability, scalability, accessibility, security.

Unit Tests:

Verify individual modules like user registration validation, password hashing, JWT generation, or KPI computation functions work correctly in isolation. Here, my focus is on core logic correctness.

Evaluate Case ID	Requirement Ref	Evaluate Scenario	Expected Result	Priority	Status
TC-01	FR 2.1	User Registration – Invalid Email	Error displayed	Critical	To Do
TC-02	FR 2.1	User Registration – Weak Password	Error displayed	Critical	To Do
TC-03	FR 2.2	User Login – Valid	JWT issued, redirected	Critical	To Do
TC-04	FR 2.2	User Login – Invalid	Error displayed; failed attempt logged	Critical	To Do
TC-05	FR 2.4	Password Reset – Valid	Password updated; old token invalidated	High	To Do
TC-06	FR 5.2	LLM Message Generation – Persona Compliance	Response aligns with persona	Critical	To Do

Integration Tests:

Here I will test interactions between components,

e.g., user messages → RAG retrieval → LLM response → session logging,

This ensures that data flows correctly across APIs, database, and AI modules.

Evaluate Case ID	Requirement Ref	Evaluate Scenario	Expected Result	Priority	Status
TC-07	FR 3.3	Target Profile Generation	Profile generated & stored; accessible to admin	Critical	To Do
TC-08	FR 4.1	Scenario Selection – Valid	Session created; chat initialized	Critical	To Do
TC-09	FR 4.2	Real-Time Chat – Send/Receive	Attacker response <2.5s	Critical	To Do
TC-10	FR 4.4	Adaptive Tactic Retrieval – Success	Tactics retrieved from Pinecone; LLM conditioned	Critical	To Do
TC-11	FR 4.5	Suspicion Detection – Keyword	Pivot tactics retrieved; event logged	High	To Do
TC-12	FR 6.1	KPI Computation	Susceptibility, leakage, latency, detection computed	Critical	To Do
TC-13	FR 7.1	Session Logging	All turns, latency, leakage, tactics logged correctly	Critical	To Do

System / End-to-End Tests:

Here I will simulate complete workflows like starting a session, interacting with AI attackers, and receiving KPI feedback.

Thus, verifying the entire platform functions for trainees and admins.

Evaluate Case ID	Requirement Ref	Evaluate Scenario	Expected Result	Priority	Status
TC-14	FR 2.1	User Registration – Valid	Account created; confirmation email sent	Critical	To Do
TC-15	FR 3.1	Personality Assessment – Complete	Dimensions scored 0–100; results saved	Critical	To Do
TC-16	FR 3.2	Technical Quiz – All Correct	Score calculated; feedback displayed	High	To Do
TC-17	FR 4.2	Real-Time Chat – Send/Receive	Chat works end-to-end	Critical	To Do
TC-18	FR 4.3	Persona Consent – Accept	Session proceeds: consent logged	High	To Do
TC-19	FR 4.6	Session Termination – User Abort	Session ends: outcome recorded	High	To Do
TC-20	FR 6.6	KPI Feedback Display	Trainee sees charts & recommendations	High	To Do

Black Box Functional Tests:

Here I will Validate platform behavior from a user perspective, such as login, scenario selection, consent flow, and adaptive attacker responses, without looking at internal implementation.

This will basically show that the system is able to be used by a user who is not aware of the functionality at all.

Evaluate Case ID	Requirement Ref	Evaluate Scenario	Expected Result	Priority	Status
TC-21	FR 3.1	Personality Assessment – Partial	Cannot skip questions	Critical	To Do
TC-22	FR 3.2	Technical Quiz – Mixed Answers	Feedback on missed questions	High	To Do
TC-23	FR 4.1	Scenario Selection – Invalid	Error displayed; choice blocked	Critical	To Do
TC-24	FR 4.3	Persona Consent – Decline	Session ended; event logged	High	To Do
TC-25	FR 4.4	Adaptive Tactic Retrieval – Failure	Fallback to local tactics	Critical	To Do
TC-26	FR 4.5	Suspicion Detection – Multiple Turns	Session auto-terminates	High	To Do
TC-27	FR 6.3	Information Leakage Detection	Leakage classified; feedback shown	High	To Do
TC-28	FR 6.5	Time to Detection	Time measured & displayed	Medium	To Do

Non-Functional Tests:

Check performance, reliability, scalability, and compliance, e.g.,

1. LLM response time <2.5s,
2. Mobile-friendly UI
3. GDPR consent handling.
4. Ensuring the system meets quality standards

These NFRs are linked to FRs of our system, hence ensure the system meets quality standards.

Evaluate Case ID	Requirement Ref	Evaluate Scenario	Expected Result	Priority	Status
TC-29	FR 9.1	Responsive Design	UI adjusts correctly on desktop/tablet/mobile	High	To Do
TC-30	FR 9.2	Accessibility Compliance	All functions accessible via keyboard/screen reader	Medium	To Do
TC-31	FR 11.1	Performance SLA	Page load & LLM response within SLA	High	To Do
TC-32	FR 11.2	Availability	System keeps core functionality under downtime	High	To Do
TC-33	FR 12.1	GDPR Compliance – Consent Withdrawal	User data remove within 30 days; audit logged	High	To Do
TC-34	FR 12.2	Audit Trail	Logs immutable; exportable for compliance	High	To Do

Implementation:

Overview of Prototype Demonstration:

The current implementation of **Aegisly** serves as a high-fidelity prototype of an AI-driven social engineering simulation platform. The demonstration highlights a full end-to-end lifecycle: from user authentication and psychological profiling to a real-time, adaptive chat simulation powered by Large Language Models (LLMs) and Retrieval-Augmented Generation (RAG).

Front-End Architecture:

The front-end is developed using **Next.js**, using the **App Router** for optimized navigation and server-side rendering.

- **State Management:** React Hooks (use State, use Effect) manage the real-time chat state, ensuring fluid message bubbles and "typing" indicators.
- **Form Handling:** The personality assessment (OCEAN model) and technical quiz are implemented as multi-step forms that confirm user input before sending data to the backend via **Server Actions**.
- **Responsive Dashboard:** A central dashboard displays user KPIs, using **Tailwind CSS** for a responsive, grid-based layout that adapts to tablet and desktop views.

Back-End and Database Integration:

The backend logic is partitioned into a modular **Node.js** environment.

- **Data Persistence:** We use **MongoDB** to store user profiles, session logs, and KPI metrics. The schema is designed to link specific psychological traits (from the assessment) to simulation outcomes.
- **Authentication:** Implementation of **NextAuth.js** (or JWT via Server Actions) ensures secure access. Password hashing is managed via **bcrypt**, and role-based access control (RBAC) separates the 'Trainee' simulation view from the 'Admin' analytics view.

AI Orchestration and RAG Pipeline:

The core of the prototype is the "Attacker Engine," orchestrated via **Lang Chain**.

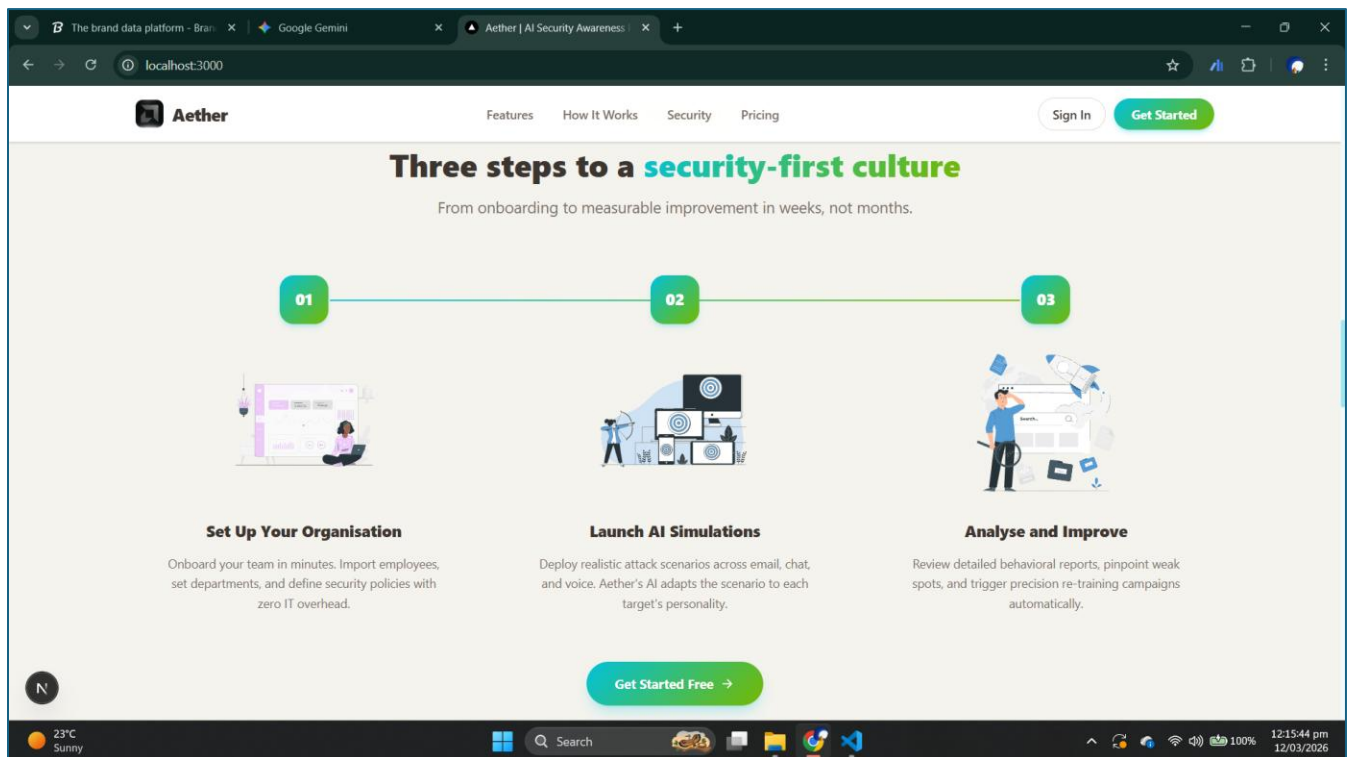
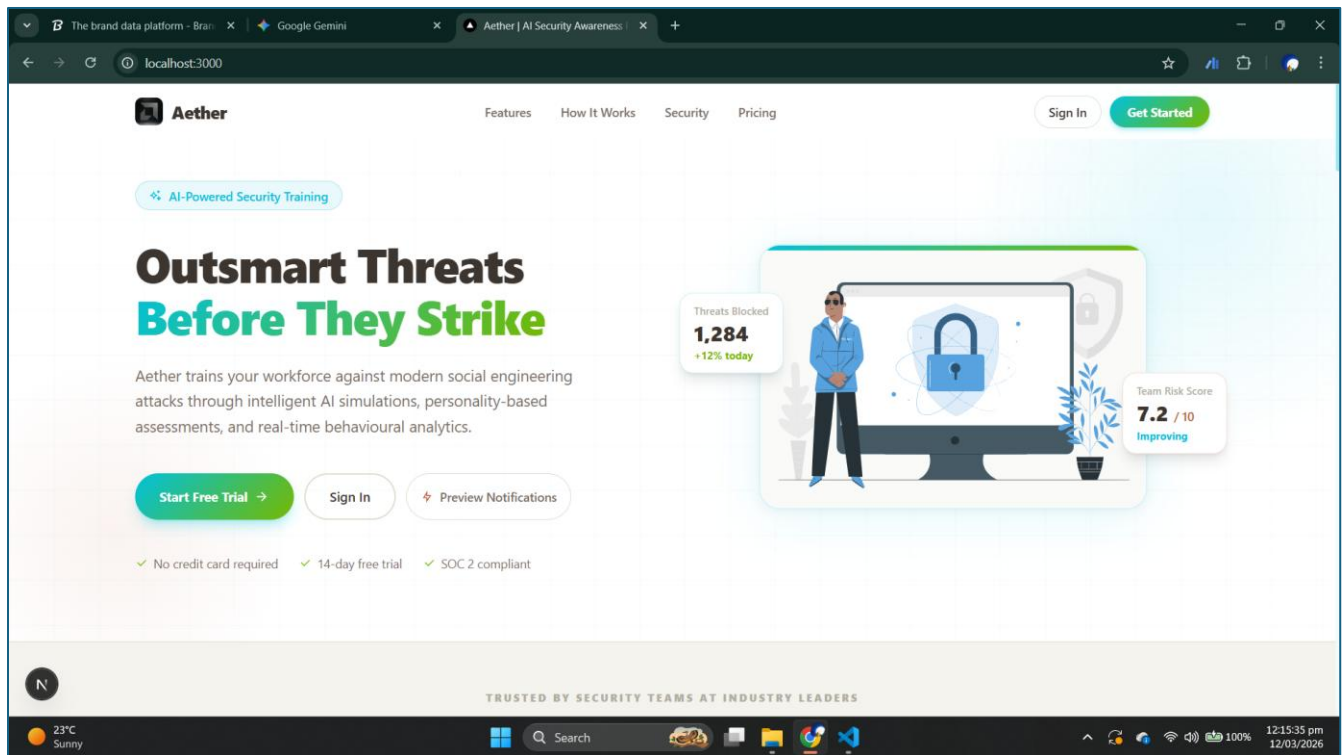
1. **Model Selection:** We use **Meta Llama 3.1 (8B)**, hosted locally via **Ollama**, to generate human-like social engineering messages.

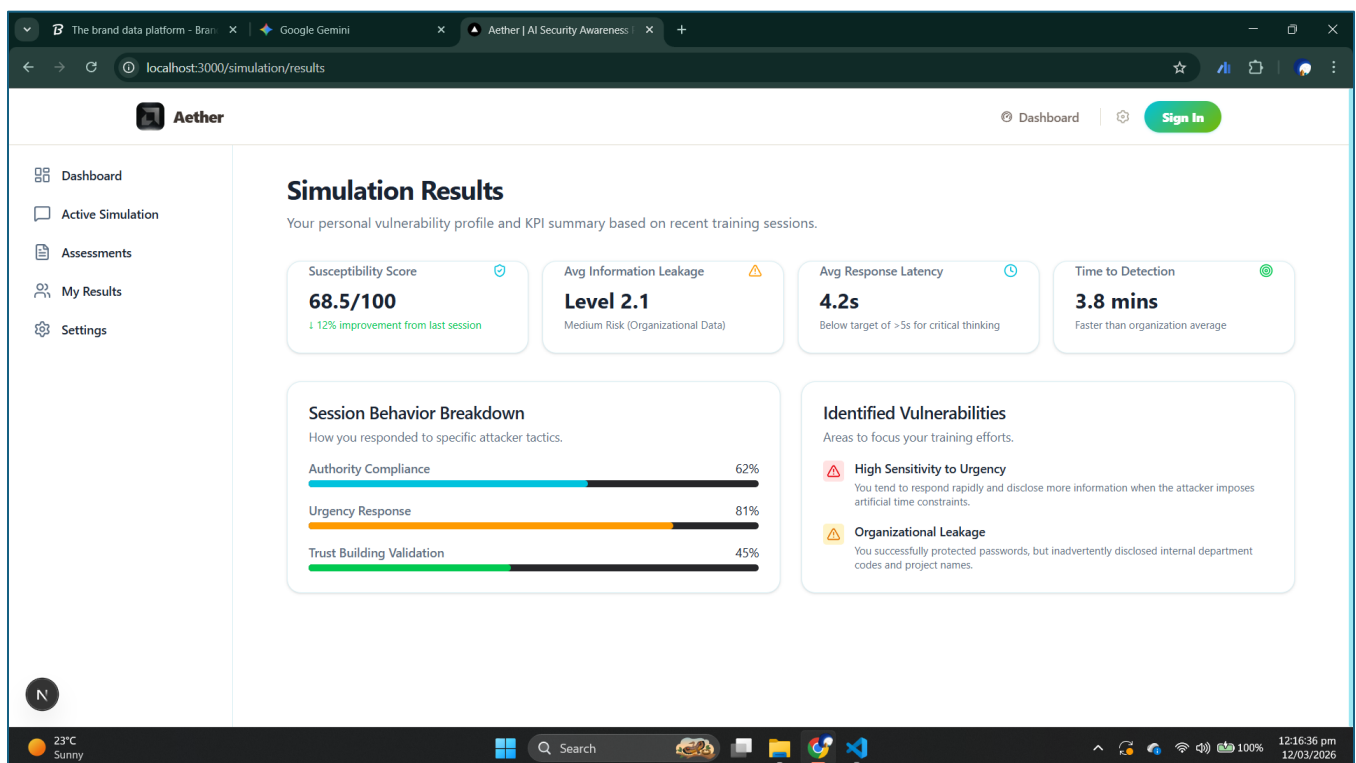
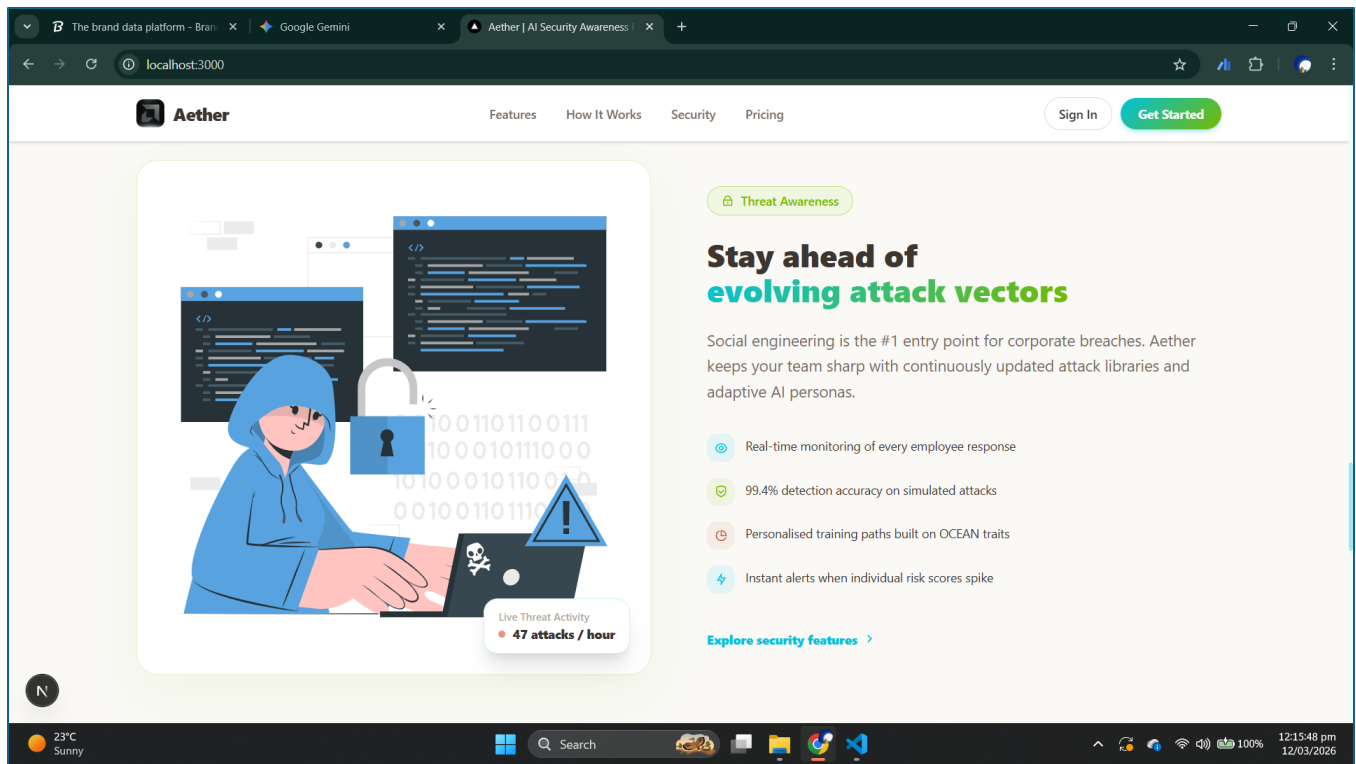
2. **RAG Mechanism:** To prevent "hallucinations" and ensure the AI follows realistic tactics, we integrated **Pinecone** as a vector database. When a user sends a message, the system generates an embedding, retrieves the most relevant social engineering tactic (e.g., "Urgency" or "Authority"), and injects it into the LLM prompt.
3. **Adaptive Logic:** The prototype proves "Pivot" capabilities. If the NLP layer detects a "Suspicion" score above **0.7**, the Lang Chain agent automatically switches tactics to a "de-escalation" or "re-engagement" strategy stored in the vector store.

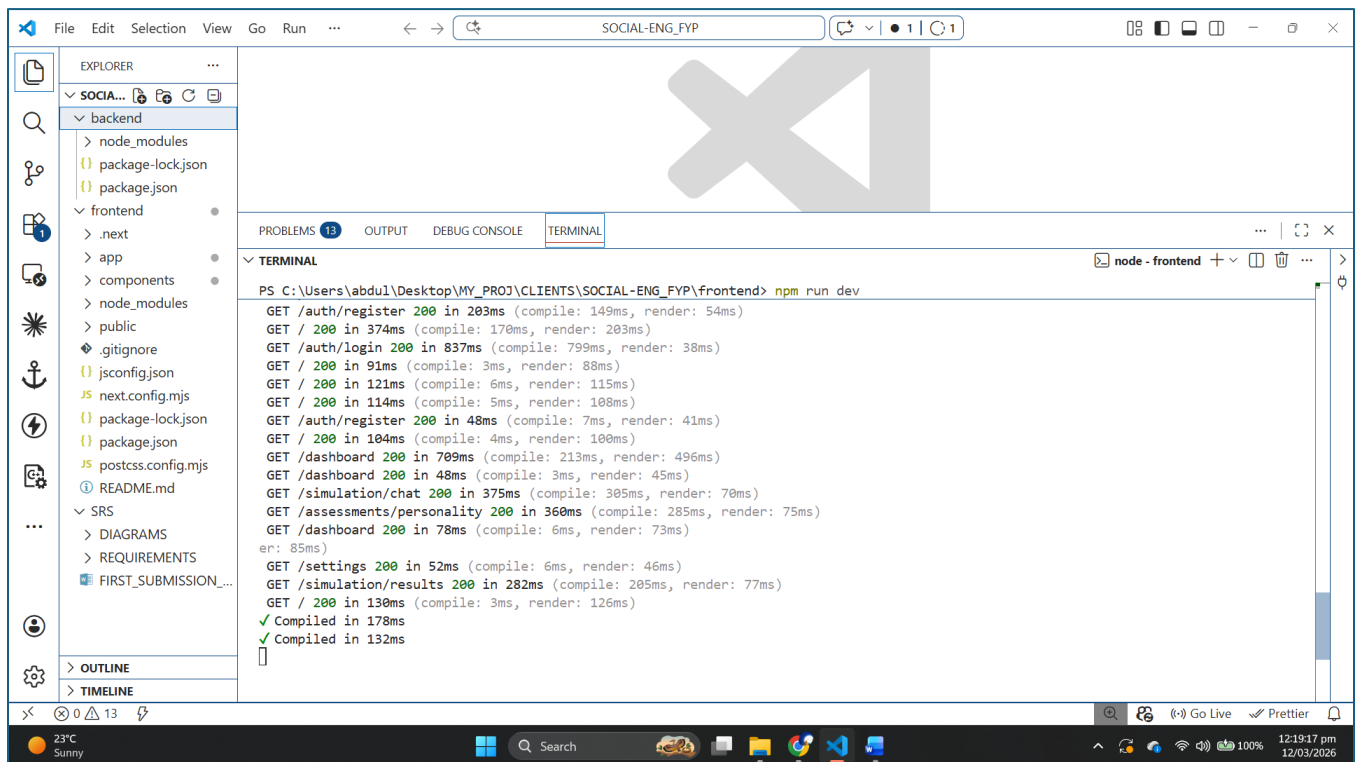
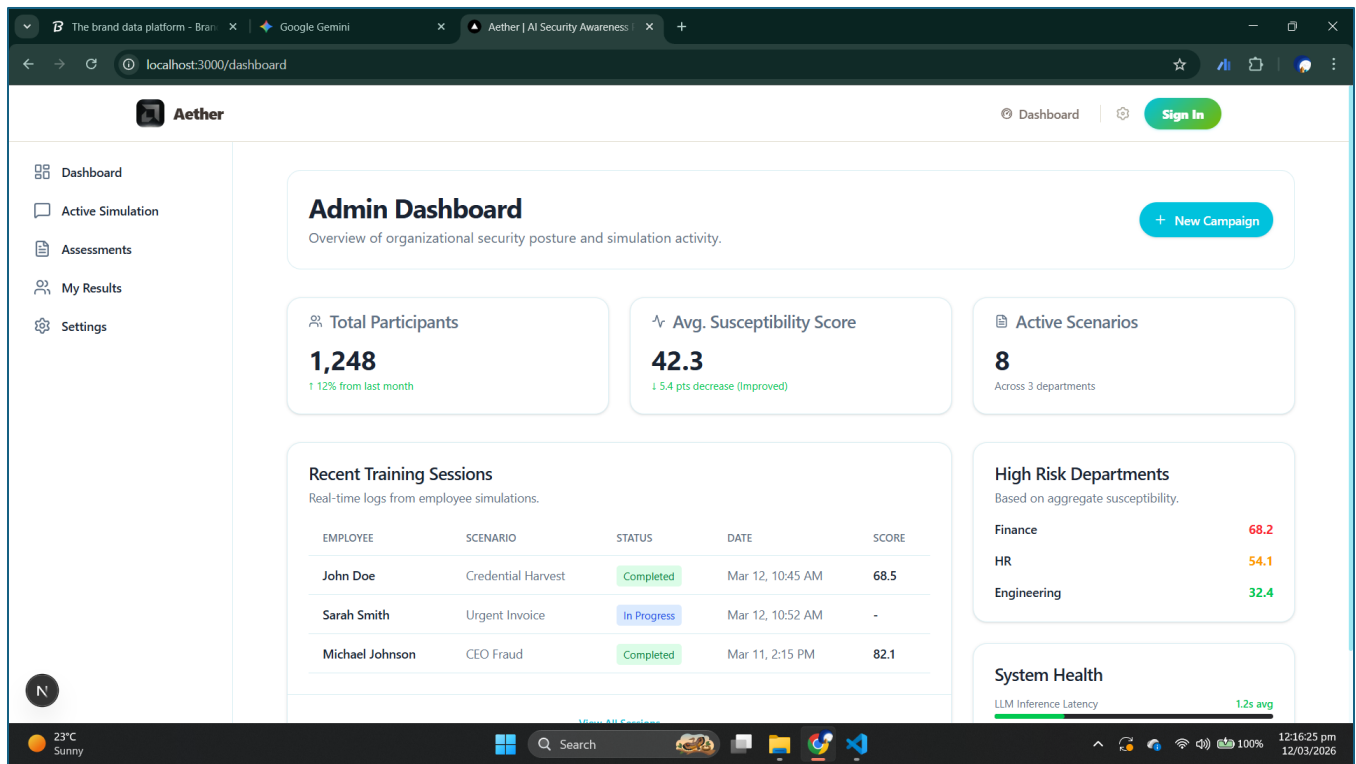
Prototype Scope for First Deliverable:

For the first demonstration, the prototype includes:

- **User Journey:** Successful registration and login flow.
- **Profiling:** Completion of the Big Five personality test with immediate scoring stored in the database.
- **Live Simulation:** A functional chat interface where a Llama-powered persona tries a "Credential Harvesting" scenario. The demo shows the AI adapting its tone based on the user's personality score.
- **Result Generation:** A post-session summary calculating "Time to Detection" and "Information Leakage" metrics.







References:

1. GeeksforGeeks. *Functional vs Non-Functional Requirements*. <https://www.geeksforgeeks.org/software-engineering/functional-vs-non-functional-requirements/>
2. GeeksforGeeks. *Unit Testing in Software Testing*. <https://www.geeksforgeeks.org/software-testing/unit-testing-software-testing/>
3. GeeksforGeeks. *Black Box Testing in Software Engineering*. <https://www.geeksforgeeks.org/software-testing/software-engineering-black-box-testing/>
4. Springer. *Software Testing & Evaluation Methods*. https://link.springer.com/chapter/10.1007/978-3-031-61382-1_5
5. ArXiv. *Recent Advances in AI-driven Security Simulations*. <https://arxiv.org/abs/2407.15912>
6. IEEE Xplore. *Next-Generation AI Security Tools*. <https://ieeexplore.ieee.org/abstract/document/10533010/>
7. Next.js Official Documentation. <https://nextjs.org/>
8. AWS. *What is Retrieval-Augmented Generation (RAG)?* <https://aws.amazon.com/what-is/retrieval-augmented-generation/>
9. Ollama Documentation. <https://docs.ollama.com/>
10. Hugging Face. *Meta Llama 3.1 8B Model*. <https://huggingface.co/meta-llama/Llama-3.1-8B>
11. Ollama Llama 3.1 Library. <https://ollama.com/library/llama3.1:8b>
12. GitHub. *Llama Stack Repository*. <https://github.com/llamastack/llama-stack>
13. Uber MFA breach. <https://centrexit.com/mfa-fatigue-uber-breach-2022/#:~:text=September%202022.,Late%20at%20night>.
14. The MGM Resorts "Vishing" Attack. <https://netwrix.com/en/resources/blog/mgm-cyber-attack/>
15. The Arup AI-Deepfake Video Scam. <https://edition.cnn.com/2024/05/16/tech/arup-deepfake-scam-loss-hong-kong-intl-hnk#:~:text=Arup%20revealed%20as%20victim%20of,Follow%20CNN%20Business>